BLE FileSync

How the SensorTile.box hands SD-card files to MovementLogger over Bluetooth — short technical breakdown. v0.1.4+ firmware, MovementLogger v0.1.4+. May 2026.

> **Get the GUI:** latest release is **MovementLogger v0.1.6** (signed + notarized + stapled, runs offline).
>
> - **macOS** (Apple Silicon): MovementLogger-0.1.6-macos-aarch64.dmg
> - **Windows** (x86_64): MovementLogger-v0.1.6-x86_64-pc-windows-msvc.zip
> - **Linux** (x86_64): MovementLogger-v0.1.6-x86_64-unknown-linux-gnu.tar.gz
> - Always-latest: github.com/zdavatz/fp-sns-stbox1/releases/latest

Why

Before BLE FileSync the only way to get sensor / GPS / battery / error-log files off the box was popping the SD card and plugging it into a reader. In the field that means screwdriver out, lose the tiny screws on the boat, swap card, screw back. BLE FileSync removes that round-trip: open the GUI, scan, connect once with a PIN, download what you want.

The two pieces

Side	Where	What it does
Firmware	`Projects/STEVAL-MKBOXPRO/Applications/Rev_C/SDDataLogFileX/Core/Src/ble_*.c`	Brings up the BlueNRG-LP, advertises as `STBoxSync`, registers two custom GATT characteristics, parses opcodes, walks the SD card, streams file bytes back over notifications.
GUI	`Utilities/rust/stbox-viz-gui/src/ble.rs` + the new "BLE FileSync" panel in `main.rs`	btleplug + tokio worker thread. Scan / connect / send opcodes / collect notifies / save files to disk. Auto-routes downloaded CSVs into the

		existing animate form.

Wire protocol

Two characteristics, both under the BlueST features service that the X-CUBE-BLEMGR middleware already registers:

Characteristic	UUID	Properties
FileCmd	`00000080-0010-11e1-ac36-0002a5d5c51b`	write w/o response
FileData	`00000040-0010-11e1-ac36-0002a5d5c51b`	notify

Opcodes (one byte + optional payload — payload is the filename without trailing NUL):

Opcode	Meaning	FileData reply
`0x01` LIST	enumerate SD root	one ASCII line per file `name,size\n`, terminator `\n`
`0x02` READ <name>	stream file body	raw bytes; total length matches the LIST size
`0x03` DELETE <name>	drop file	single status byte
`0x04` STOP_LOG	gracefully close active session	none — host re-checks via LIST

Status bytes used by READ / DELETE replies: `0x00` OK, `0xB0` BUSY, `0xE1` NOT_FOUND, `0xE2` IO_ERROR, `0xE3` BAD_REQUEST. READ and DELETE are rejected with `BUSY` while a `Sens*.csv` or `Gps*.csv` is open for writing — host calls STOP_LOG first.

Firmware-side architecture

One ThreadX thread at priority 14 owns the BLE stack. Below the FileX writer (12) and the GPS thread (~11) so SD bandwidth and GPS line assembly always win — BLE never starves the logger.

```
// Core/Src/ble_sync.c — the entire BLE thread loop
for (;;)
{
    if (hci_event) { hci_event = 0; hci_user_evt_proc(); }
    BleFileSync_Tick();           // advance LIST / READ state machine
    tx_thread_sleep(1);           // 10 ms
}
```

The opcode handler (`ble_filesync.c`) is a small state machine. `write_request_filecmd()` runs inside the HCI callback, parses the opcode, sets the next state. `BleFileSync_Tick()` runs at thread priority outside the callback and does the work — FileX directory walks, file reads, notify writes — so the HCI event pump never blocks. On notify congestion we don't drop bytes: LIST stays in `ST_LIST_EMIT` and READ rewinds with `fx_file_relative_seek(SEEK_BACK)`, both retry on the next tick.

Module map:

- `ble_sync.c` — owns the ThreadX thread; calls `bluetooth_init()` once then loops the HCI / Tick pair shown above.
- `ble_filesync.c` — the FileCmd/FileData characteristics + the LIST / READ / DELETE / STOP_LOG state machine.
- `ble_implementation.c`, `ble_function.c` — minimal X-CUBE-BLEMGR glue (mandatory globals + `set_board_name` + connection / pair callbacks).
- `ble_spi.c`, `hci_tl_interface.c` — isolated SPI1 driver bypassing the shared `SensorTileBoxPro` BSP. The BLEDualProgram BSP and the SDDataLogFileX BSP both export the same `BSP_*` symbols and can't coexist; bypassing the BSP for the BLE link sidesteps the collision and keeps the stack debuggable in isolation.
- `FileX/App/app_filex.c` — gains two public hooks: `Ble_RequestStopLog()` posts `COMMAND_STOP_LOG` into the FileX writer's `MessageQueue`; `Ble_IsLoggingActive()` tells the BUSY guard whether `SensorsFileOpen` or `GpsFileOpen`.

Build cost vs the no-BLE build: text +28.7 KB, BSS +4.5 KB. Gated on `STBOX1_ENABLE_BLE_SYNC 1` in `stbox1_config.h` (default on); flip to 0 to remove the entire stack.

GUI-side architecture

The egui UI is sync, btleplug is async — bridge with one tokio current-thread runtime on a dedicated worker thread. Commands and events shuttle across two `std::sync::mpsc` channels. The UI drains events on every frame; the worker awaits inside the runtime.

The key design decision: **one notification stream per connection, not per op.** btleplug's `Peripheral::notifications()` returns a stream from "subscription start". Opening a fresh one per LIST / READ risks losing the first row / chunk if the box notifies before the `await` is parked. So the stream is opened once on connect and demuxed inside a single `tokio::select!` over the command channel + the stream + a 200 ms watchdog tick:

```
tokio::select! {
    cmd   = arx.recv()      => state.handle_command(cmd).await,
    notif = stream.next()   => state.handle_notification(notif).await,
    _     = &mut watchdog   => state.tick_watchdog(),
}
```

Notifications get dispatched into whichever op is in flight (`CurrentOp::Listing` / `CurrentOp::Reading` / `CurrentOp::Idle`). Disconnect mid-op tears down the peripheral and surfaces an explicit error like `READ Sens005.csv aborted by disconnect at 1234/5678 B` — no orphan worker spinning on a dead stream. The watchdog wakes every 200 ms so a real 20 s notify drought (link drop, stuck firmware) surfaces a timeout error instead of hanging.

READ status-byte detection is a small but important detail: a single-byte first notify is only treated as an error when the byte is in `{0xB0, 0xE1, 0xE2, 0xE3}`. A 1-byte CSV / log file (whose lone byte is plain ASCII well below 0x80) streams correctly because no legitimate file ever starts with one of those high-range status codes.

UI flow

Collapsible "BLE FileSync" panel between the file summary and the animation parameters:

1. **Scan** — 5 s scan, lists every `STBoxSync` peripheral with RSSI.

2. **Connect** — first time the OS pops a Bluetooth permission prompt and a pairing dialog (PIN `123456`).
3. **Refresh file list** — sends LIST, populates a checklist of `name + size + checkbox`. All rows pre-ticked.
4. **Download selected** — for each ticked file: send READ, accumulate raw bytes until the LIST size is reached, write to the chosen output folder (default `csv/`).
5. Files matching `Sens*.csv` or `*_gps.csv` auto-route into the form's Sensor / GPS slots so the user can hit Generate without re-dragging.
6. **STOP_LOG** — needed if the box is mid-session; READ refuses to read currently-open log files with `BUSY`.

Platform notes

- **macOS**: needs Bluetooth consent. The bundled `.app` carries `NSBluetoothAlwaysUsageDescription` in `Info.plist`, and the App-Sandbox build adds `com.apple.security.device.bluetooth` in the entitlements plist. A bare `cargo run` on a fresh user account may not trigger the prompt and btleplug will silently report no adapter — install the `.app` for the consent flow.
- **Linux**: BlueZ via DBus; user must be in the `bluetooth` group. The release workflow apt-installs `libdbus-1-dev` for btleplug's BlueZ backend.
- **Windows**: WinRT BLE, no extra setup.

> **Relation to the SD-card update path:** BLE FileSync only *reads* from the card (and optionally DELETEs). Firmware updates still go via DFU (preferred) or by dropping `firmware.bin` on the SD root — those paths haven't changed.